

Smart Plant Monitoring and Watering System

**Project report submitted as the Major Test for
Integrated Training Program on Rapid
Prototyping on Embedded System and
Automation**

By
Shaik Afreen
Enrollment Number: 419821.

Organised by



**India Space Academy,
Department of Space Education, India
Space Week**

TITLE

Smart Plant Monitoring and Watering System.

PROBLEM STATEMENT

Indoor plants often require regular monitoring and timely watering to maintain healthy growth. Many plant owners may forget to water their plants, be unaware of changing environmental conditions, or have difficulty monitoring plants while away from home. Traditional plant care methods also lack real-time feedback and user interaction.

This project aims to develop a Smart Plant Monitoring & Watering System using ESP32 that automates irrigation, monitors environmental conditions, tracks water availability, and provides remote monitoring through IoT technologies. The system also incorporates interactive features such as proximity detection and touch-based expression changes to create a more engaging and user-friendly plant care experience.

COMPONENTS USED

S.No	Component	Quantity	Purpose
1.	ESP32 Development Board	1	Main controller responsible for sensor interfacing, automation, and IoT communication
2.	Soil Moisture Sensor	1	Measures soil moisture level and determines watering requirements
3.	DHT11 Sensor	1	Measures ambient temperature and humidity
4.	HC-SR04 Ultrasonic Sensor	1	Monitors water level inside the storage tank
5.	SSD1306 OLED Display	1	Displays sensor readings and animated expressions
6.	Relay Module	1	Controls the water pump through ESP32
7.	Mini Water Pump (9 V)	1	Supplies water to the plant when required
8.	Active Buzzer	1	Alerts the user when the water tank level becomes low

9.	IR Proximity Sensor	1	Detects the presence of a nearby user
10.	Touch Sensor (TTP223)	1	Allows interaction and expression changes on the OLED display
11.	4 channel bi directional level shifter	1	Provides voltage level compatibility between devices operating at different logic levels
12.	Water Tank/Container	1	Stores water for automatic irrigation
13.	Water Tube/Pipe	1	Delivers water from the tank to the plant
14.	Jumper Wires	1	Electrical connections between components
15.	9V battery	1	Used to power the water pump

CIRCUIT DIAGRAM

The circuit diagram of the Smart Plant Monitoring and Watering System is shown below. The ESP32 acts as the central controller and interfaces with the soil moisture sensor, DHT11 sensor, HC-SR04 ultrasonic sensor, IR sensor, touch sensor, OLED display, relay module, buzzer, and water pump. A logic level converter is used to ensure voltage compatibility between components operating at different logic levels.

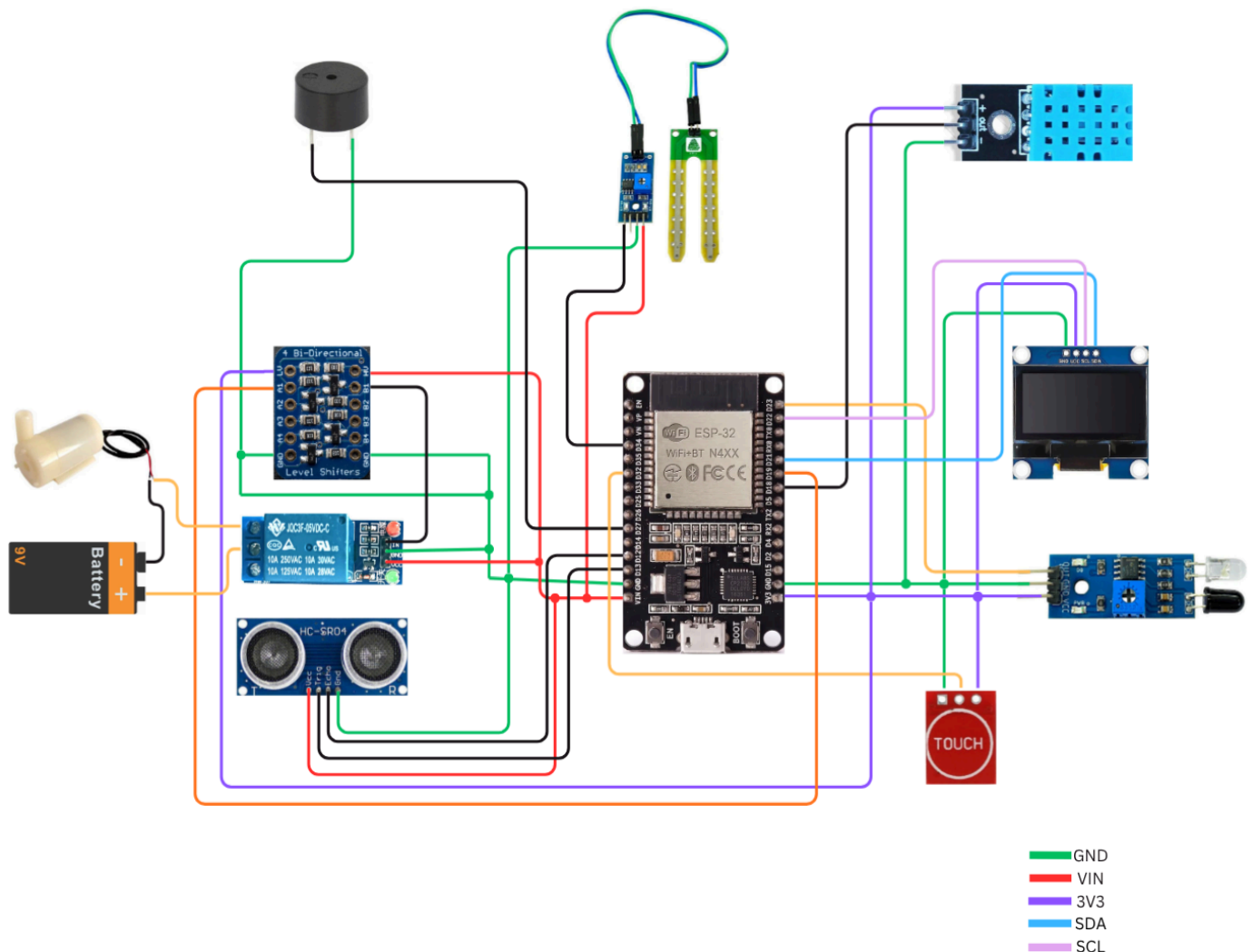


Figure 1. Hardware Circuit Diagram of the Smart Plant Monitoring and Watering System.

Detailed Hardware Connections

Soil Moisture Sensor

Sensor Pin	ESP32 Connection
AO	GPIO 34
VCC	VIN
GND	GND

Touch Sensor

Sensor Pin	ESP32 Connection
I/O	GPIO 32
VCC	3V3
GND	GND

Buzzer

Buzzer Pin	ESP32 Connection
Positive (+)	GPIO 27
Negative (-)	GND

HC-SR04 Ultrasonic Sensor

Sensor Pin	ESP32 Connection
TRIG	GPIO 13
ECHO	GPIO 12
VCC	VIN
GND	GND

IR Proximity Sensor

Sensor Pin	ESP32 Connection
OUT	GPIO 23
VCC	3V3
GND	GND

DHT11 Temperature & Humidity Sensor

Sensor Pin	ESP32 Connection
DATA	GPIO 18
VCC	VIN
GND	GND

OLED Display (SSD1306)

OLED Pin	ESP32 Connection
SCL	GPIO 22
SDA	GPIO 21
VCC	3V3
GND	GND

4-Channel Bi-Directional Level Shifter

Low Voltage

Level Shifter Pin	ESP32 Connection
LV	3V3
GND	GND
A1	GPIO 19

High Voltage Side

Level Shifter Pin	ESP32 Connection
HV	VIN
GND	GND
B1	Relay IN

Relay Module Connections

Relay Pin	Connection
IN	B1 of Logic Level Converter
VCC	HV Supply from Level Shifter (5V/VIN)
GND	GND

Water Pump and Battery Connections

9V Battery

Battery Terminal	Connection
Positive (+)	Relay COM Terminal
Negative (-)	Water Pump Negative Terminal

Water Pump

Pump Wire	Connection
Positive (+)	Relay NO (Normally Open) Terminal
Negative (-)	Battery Negative Terminal

Relay Power Path

Relay Terminal	Connection
COM	Battery Positive (+)
NO (Normally Open)	Water Pump Positive (+)
NC (Normally Closed)	Not Connected

PCB DESIGN FILES

PCB Design Process

Step 1: Schematic Design

The schematic was created using EasyEDA's component library. Components such as the ESP32, soil moisture sensor, DHT11 sensor, HC-SR04 ultrasonic sensor, IR sensor, touch sensor, OLED display, relay module, buzzer, and logic level converter were selected from the available libraries with appropriate footprints.

To improve readability and reduce excessive wire crossings, net ports were used to establish connections between components. This resulted in a cleaner and more organized schematic design.

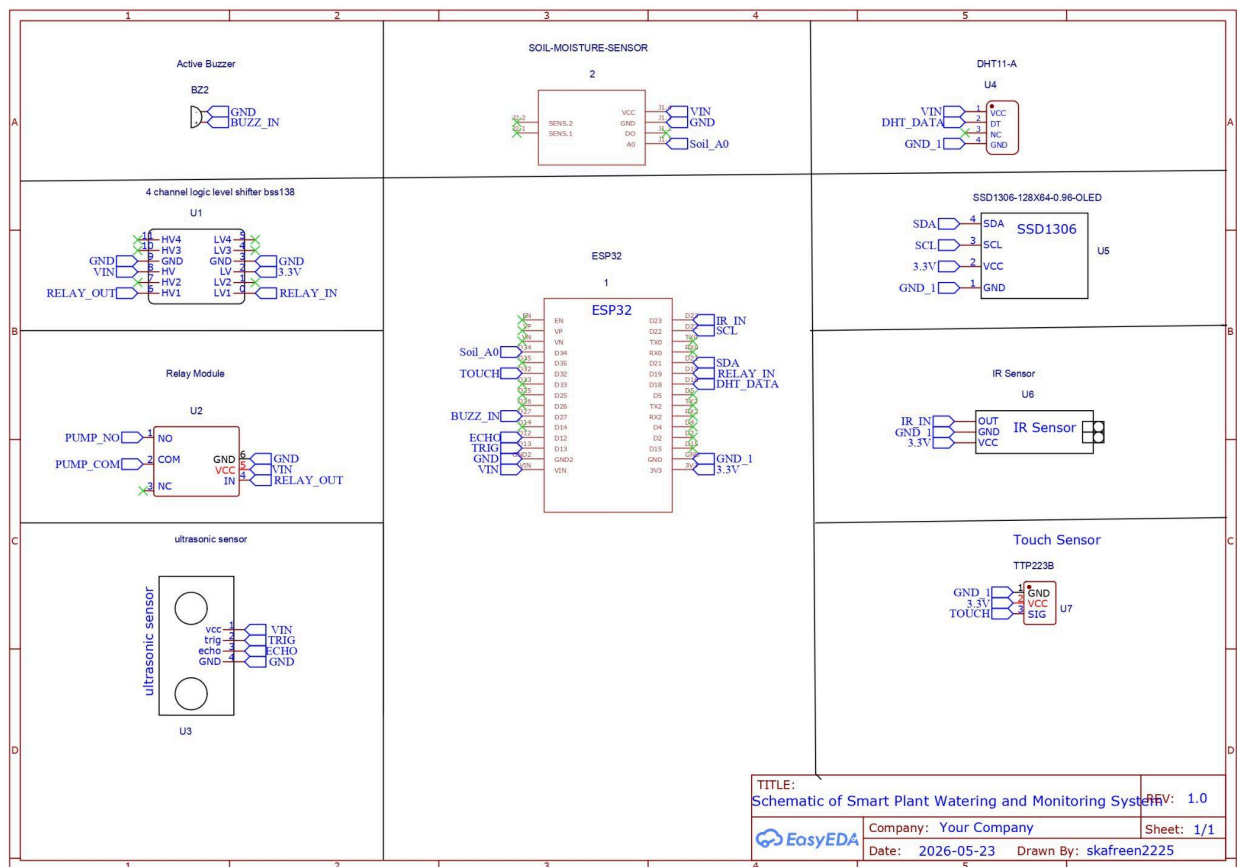


Figure1: Schematic Design of the Smart Plant Monitoring and Watering System

Step 2: PCB Layout Design

After completing the schematic, the design was converted into a PCB layout using the **Design** → **Convert to PCB** option in EasyEDA. Component footprints were arranged carefully to optimize routing and improve board organization.

Copper tracks were routed using both the top and bottom layers to prevent overlapping connections and reduce routing congestion. The Track tool was used to establish electrical connections between components.

Once routing was completed, a Design Rule Check (DRC) was performed to identify and eliminate design rule violations, ensuring that the PCB met manufacturing and electrical design requirements.

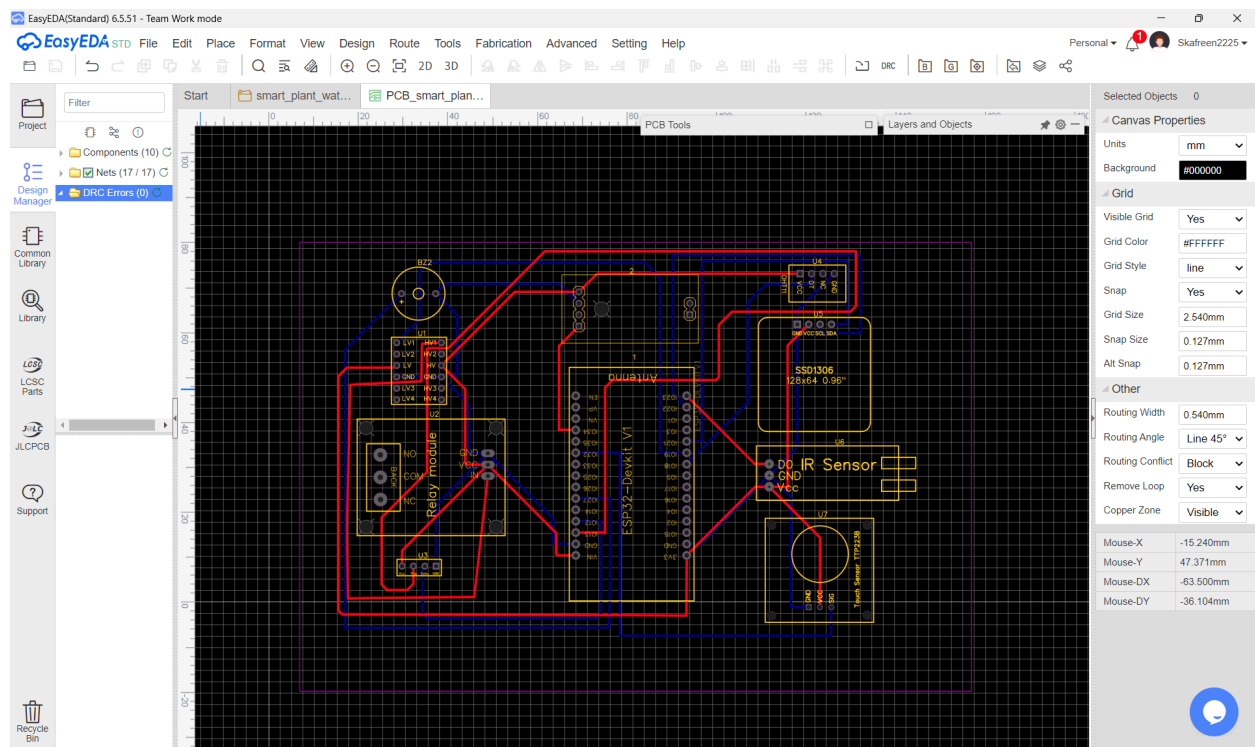


Figure 2: PCB Layout of the Smart Plant Monitoring and Watering System.

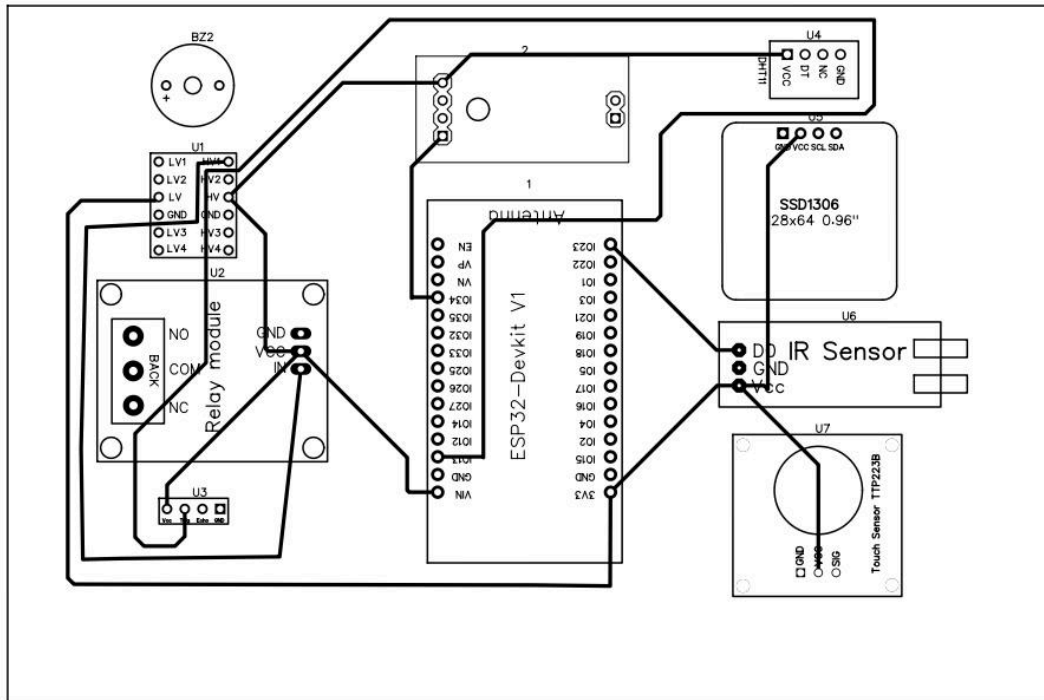


Figure 3: TOP Layer of the PCB design.

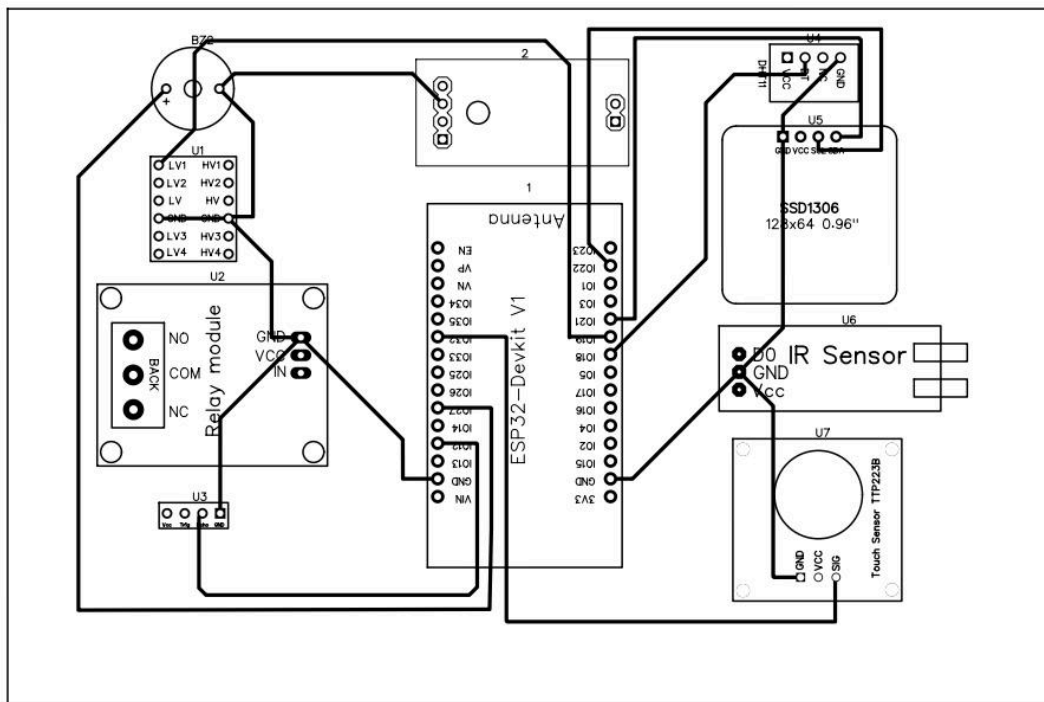


Figure 4: BOTTOM Layer of the PCB design.

Step 3: 3D Visualization

The completed PCB was visualized using the **View** → **3D View** feature in EasyEDA. This provided a realistic representation of component placement and overall board appearance before fabrication.

The 3D view helped verify component orientation, spacing, and the overall physical structure of the PCB.

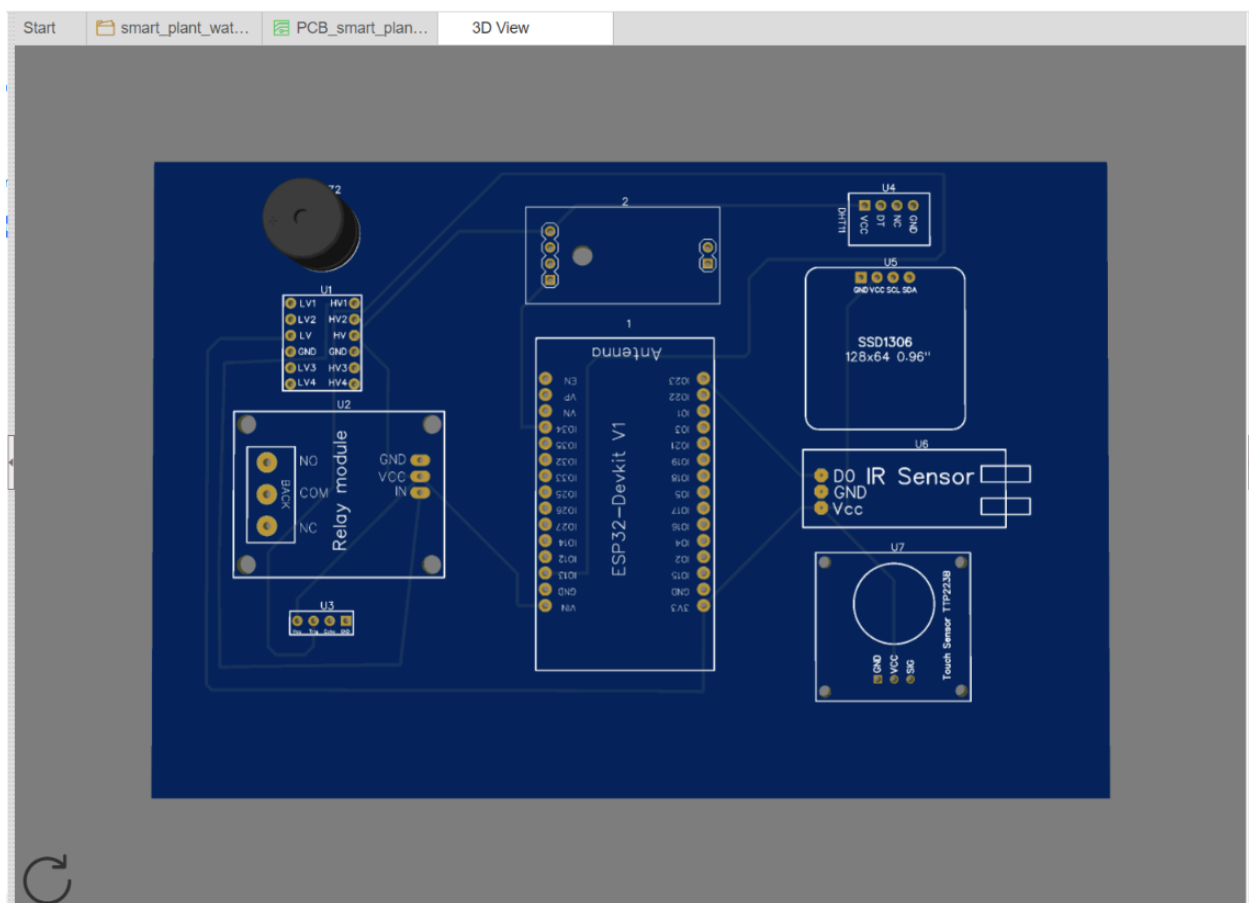


Figure 5: 3D View of the PCB design (Front).

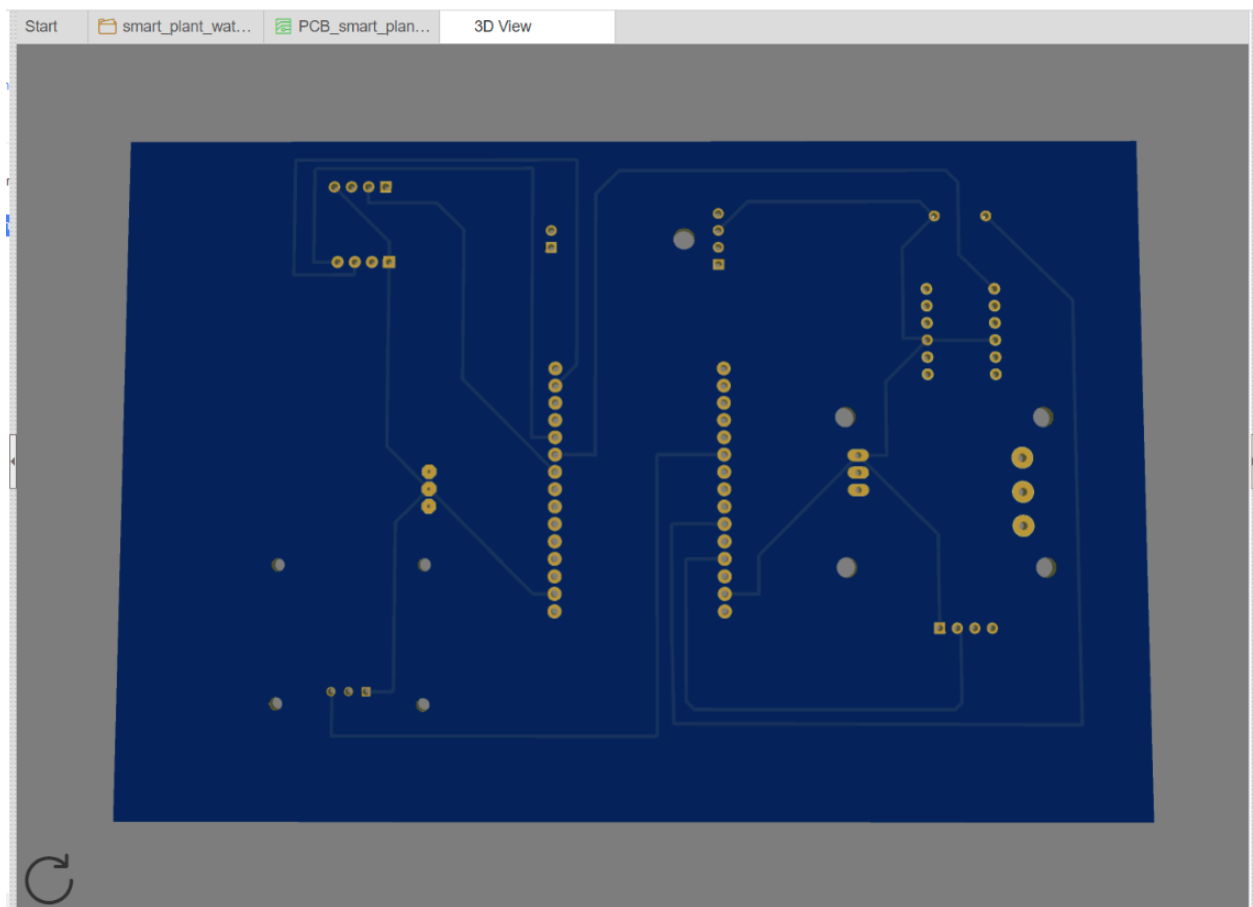


Figure 6: 3D View of the PCB design (Back).

CAD DESIGN FILES

The enclosure for the Smart Plant Monitoring & Watering System was designed using Onshape CAD software. The objective was to create a compact and organized structure capable of housing the plant, water reservoir, electronics, sensors, and user interaction components within a single enclosure.

CAD Design Process

The design began with the creation of the main enclosure, which consists of a rectangular shell with dimensions of approximately 160 mm × 250 mm × 260 mm. The enclosure was divided into dedicated compartments for plant growth, electronics, and water storage to improve accessibility and maintenance.

A separate sliding door was designed to provide access to the electronics compartment, while a removable drawer was incorporated at the bottom of the enclosure to serve as the water reservoir. Individual components were modeled separately and later assembled using Onshape assembly tools and slider mates to simulate the movement of the sliding door and water drawer.

Design Features

Plant Section

The upper-left section of the enclosure is dedicated to the plant holder. To prevent water accumulation around the plant roots, the base of this compartment was designed with a 4 mm slope that guides excess water toward the drainage area.

Additionally, three drainage holes were incorporated at the bottom of the plant compartment to allow excess water to flow out efficiently. These holes work in conjunction with the sloped base and drainage system to prevent waterlogging, improve root health, and facilitate the removal of excess irrigation water.

Electronics Compartment

The upper-right section of the enclosure houses the ESP32, sensors, relay module, and associated circuitry. A sliding access door was provided to simplify maintenance and future upgrades.

An enclosure for the ultrasonic sensor was integrated into the bottom of this compartment, allowing the sensor to face downward toward the water reservoir for accurate water-level measurement.

Ventilation gaps were incorporated into the electronics compartment to facilitate airflow around the electronic components. These openings help dissipate heat generated during operation, reducing the risk of overheating and improving the overall reliability of the system.

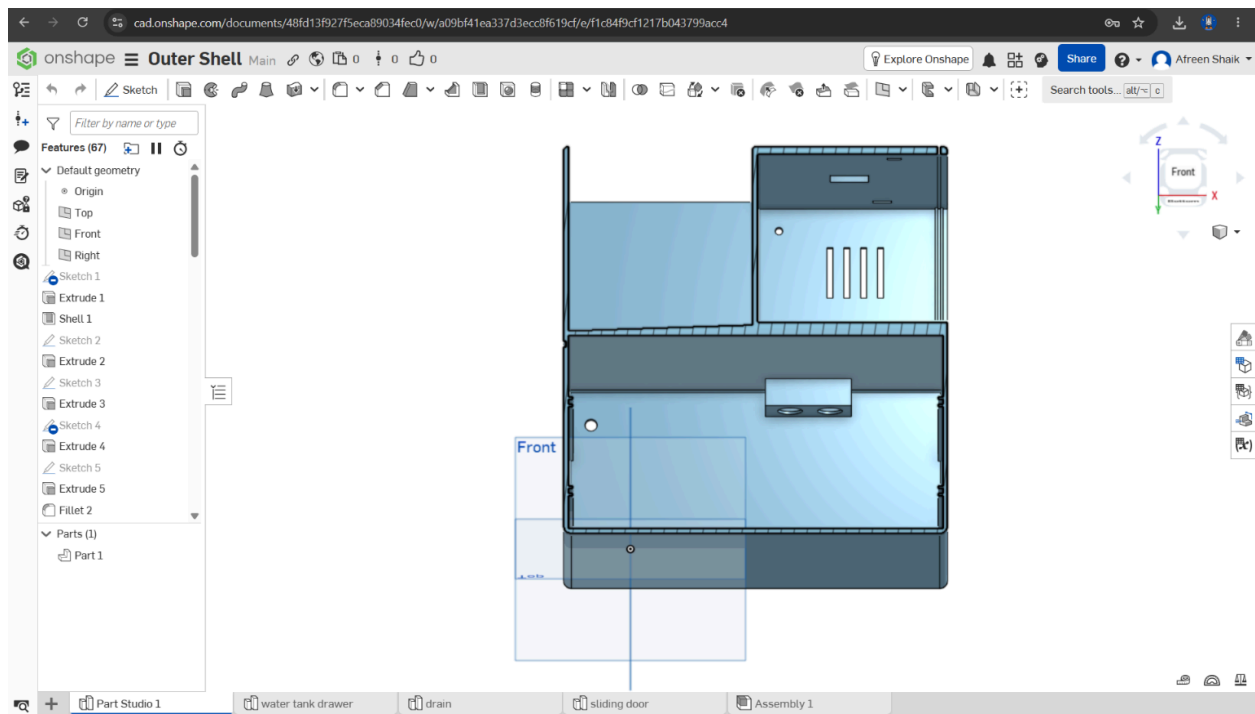


Figure 7: Image depicting Ultrasonic Sensor Enclosure at the bottom along with ventilating gaps at the side(Front sectional View).

Water Reservoir

A drawer-type water tank was designed at the bottom of the enclosure. The sliding drawer mechanism allows convenient removal and refilling of the water reservoir without disturbing the plant or electronic components.

Drainage System

A removable drainage component was incorporated beneath the plant compartment to collect and direct excess water away from the plant. This design helps prevent overwatering and improves water management within the enclosure.

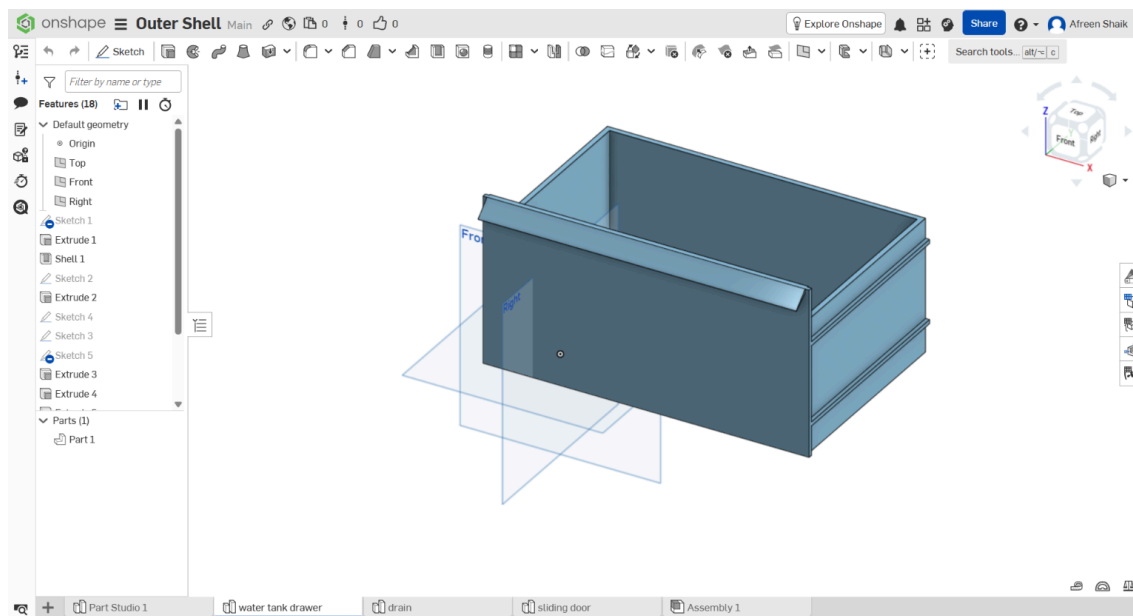


Figure 8: Image depicting CAD design of the Water Tank Drawer.

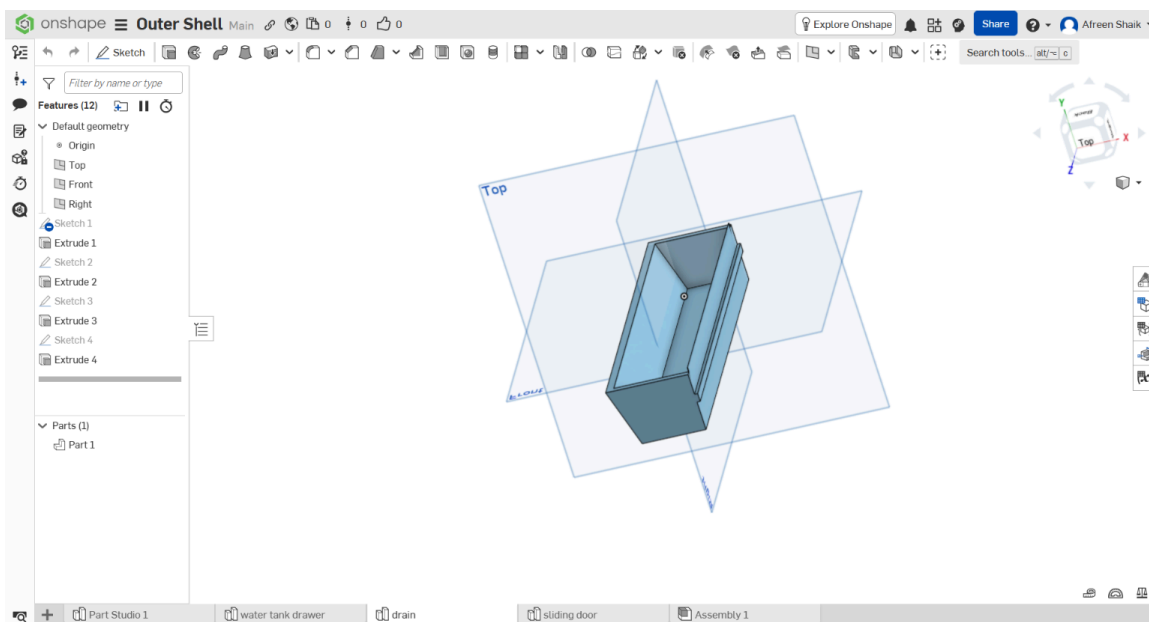


Figure 9: Image depicting CAD design of the Drain Slot.

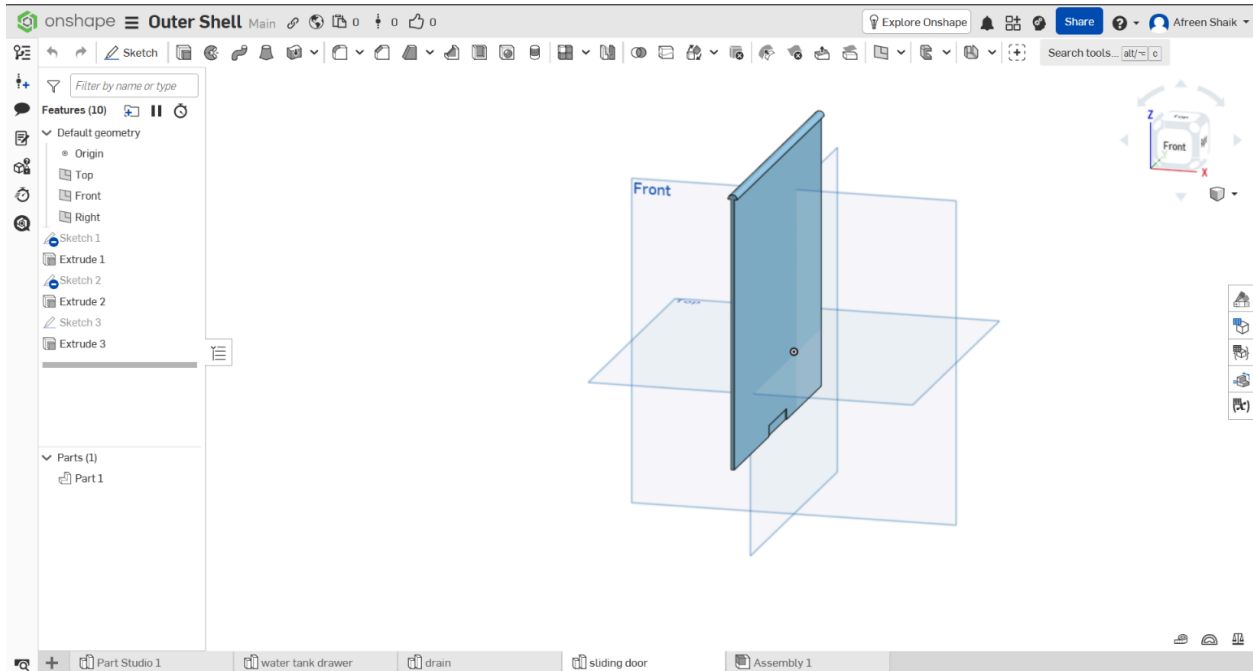


Figure 10: Image depicting CAD design of the Sliding Door of the Electronics Compartment.

Sensor Placement

Dedicated mounting provisions were incorporated for all sensors:

- OLED display opening on the top surface for user visibility.
- IR sensor opening on the front panel for proximity detection.
- Touch sensor opening on the top surface for user interaction.
- DHT11 sensor shelter on the top portion of the enclosure to enable accurate temperature and humidity measurement while providing basic protection.
- Rear opening for routing the water delivery tube from the reservoir to the plant compartment.

Assembly

The complete design was assembled in Onshape using individual parts, including the main enclosure, sliding electronics door, water reservoir drawer, and drainage component. Assembly constraints and slider mates were used to validate the motion of movable components and ensure proper fit within the enclosure.

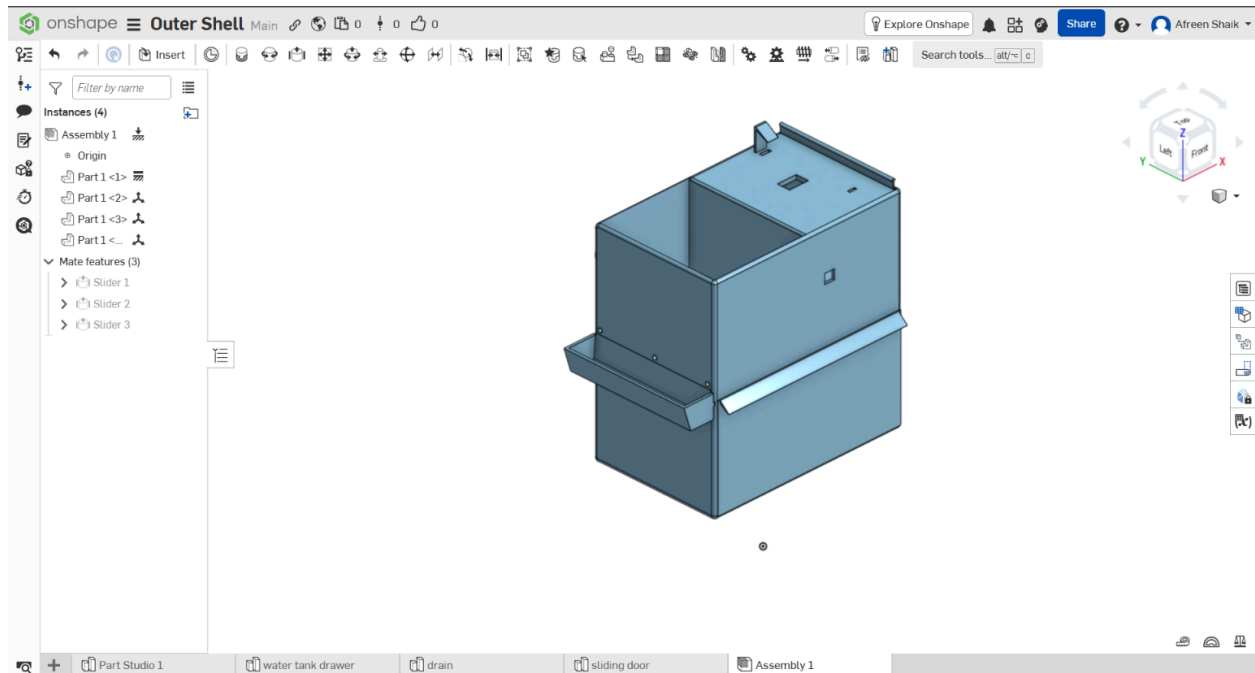


Figure 11: Image depicting CAD design of the Complete enclosure for Smart Plant Monitoring and Watering.

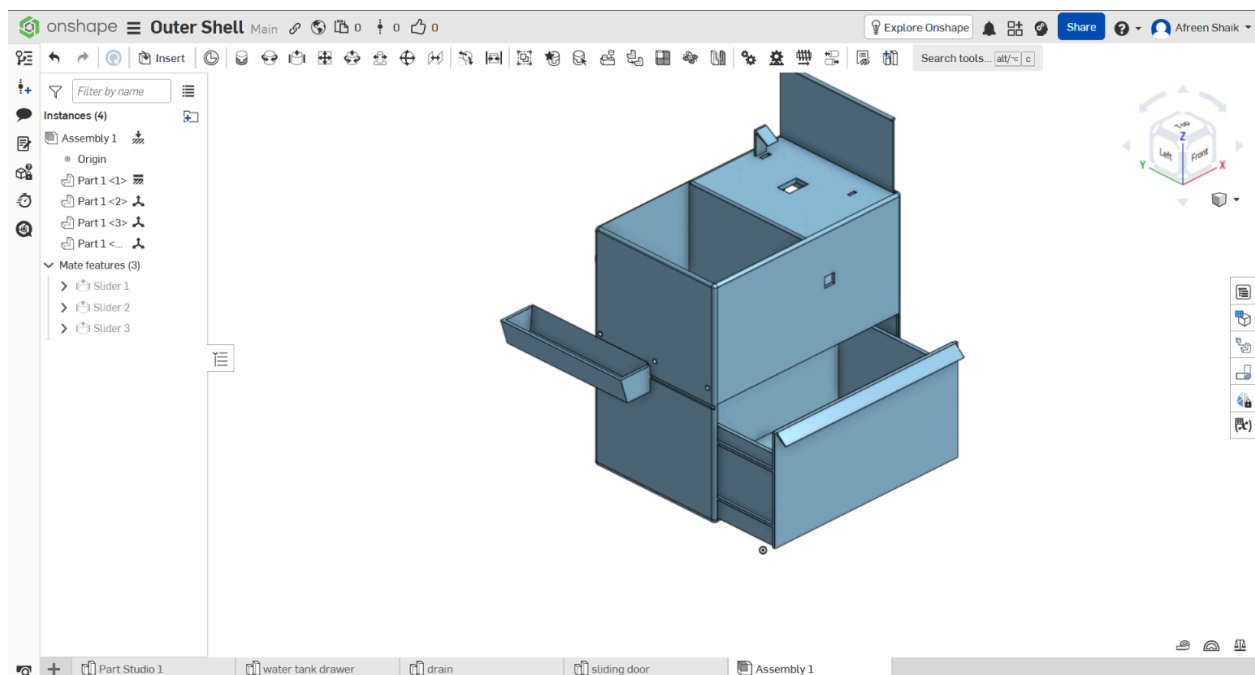


Figure 12: Image depicting Sliding of Drain, Drawer and Door.

SOURCE CODE

The Smart Plant Monitoring & Watering System was programmed using the Arduino IDE and implemented on the ESP32 microcontroller. The source code is responsible for reading sensor data, controlling the water pump, monitoring the water tank level, displaying information on the OLED screen, and managing user interactions through the touch and IR sensors.

Two software implementations were developed during this project:

- ESP32 Implementation – Performs local monitoring, automatic irrigation, water-level detection, OLED dashboard display, and interactive OLED expressions.
- Blynk IoT Implementation – Extends the system with remote monitoring capabilities by transmitting sensor data to a Blynk dashboard through Wi-Fi connectivity.

ESP32 Implementation:

```
#include <Wire.h>

#include <Adafruit_GFX.h>

#include <Adafruit_SSD1306.h>

#include <DHT.h>

#define TOUCH_ACTIVE_STATE    HIGH

#define IR_ACTIVE_STATE      LOW

#define SOIL_PIN              34

#define RELAY_PIN              19

#define TRIG_PIN              13

#define ECHO_PIN              12

#define BUZZER_PIN            27

#define DHTPIN                18
```

```

#define DHTTYPE      DHT11

#define IR_PIN       23

#define TOUCH_PIN    32

#define SCREEN_WIDTH 128

#define SCREEN_HEIGHT 64

#define OLED_RESET   -1

Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire,
OLED_RESET);

DHT dht(DHTPIN, DHTTYPE);

// _____

//  THRESHOLDS

// _____

int dryThreshold    = 3000;

int moistThreshold = 1800; // Increased from 1200 to accommodate
typical ESP32 capacitive sensor ranges

int waterThreshold = 8;    // cm - distance above this = tank empty

// _____

//  SENSOR VALUES & FILTER STATS

// _____

int  soilValue      = 0;

float temperature    = 0.0;

float humidity       = 0.0;

float waterDistance  = 0.0;

bool  pumpRunning    = false;

```

```

// Glitch-filtering state for HC-SR04

int consecutiveWaterFailures = 0;

#define WATER_FAILURE_LIMIT      3 // Require 3 consecutive
failed/empty readings to alert/stop pump

#define ULTRASONIC_READ_SAMPLES  3 // Number of hardware pulses to
average per cycle

// -----

// TIMING & DEBOUNCE

// -----

unsigned long lastSensorRead = 0;

unsigned long lastDHTRead      = 0;

#define SENSOR_INTERVAL 2000

#define DHT_INTERVAL     4000

unsigned long lastTouchTime    = 0;

unsigned long lastIRTime       = 0;

#define TOUCH_DEBOUNCE_MS     300

#define IR_DEBOUNCE_MS        300

// Cooldown to lock out triggers after an animation finishes,
guaranteeing dashboard visibility

unsigned long lastAnimationEndTime = 0;

bool          cooldownActive      = false;

#define COOLDOWN_DURATION      5000 // 5 seconds dashboard display
guarantee

// -----

// EYE SEQUENCE STATE

```

```

// -----

bool        eyeActive        = false;    // true while an eye
sequence is playing

bool        showingEyes      = false;    // true during the 3s eye
phase

bool        showingText      = false;    // true during the 2s text
phase

unsigned long eyePhaseStart   = 0;

int         currentExprIdx    = 0;        // 0-2, which expression is
playing

String      currentMessage    = "";

String      currentTrigger    = "";        // "touch" or "ir"

#define EYE_DURATION 3000

#define TEXT_DURATION 2000

// -----

//  SHUFFLED DECK – shared structure for touch & IR

// -----

struct Deck {

    int order[3];        // shuffled order of indices 0,1,2

    int pos;              // current position in deck (0,1,2)

    int lastPlayed;       // last expression index played (-1 = none yet)

};

Deck touchDeck;

Deck irDeck;

// Fisher-Yates shuffle, ensuring deck[0] != lastPlayed

```



```

void shuffleDeck(Deck &d) {

    d.order[0] = 0; d.order[1] = 1; d.order[2] = 2;

    for (int i = 2; i > 0; i--) {

        int j = random(0, i + 1);

        int tmp = d.order[i];

        d.order[i] = d.order[j];

        d.order[j] = tmp;

    }

    if (d.lastPlayed != -1 && d.order[0] == d.lastPlayed) {

        int swapWith = random(1, 3); // 1 or 2

        int tmp = d.order[0];

        d.order[0] = d.order[swapWith];

        d.order[swapWith] = tmp;

    }

    d.pos = 0;

}

int nextCard(Deck &d) {

    if (d.pos >= 3) shuffleDeck(d);

    int expr = d.order[d.pos];

    d.pos++;

    d.lastPlayed = expr;

    return expr;

}

```

```
// -----  
  
//  EXPRESSION DATA  
  
// -----  
  
// Touch expressions: 0=Grumpy, 1=Dizzy, 2=Sleepy  
  
const char* touchMessages[3] = {  
  
    "Hey! I was asleep!",  
  
    "Whoa... dizzy!",  
  
    "Zzz... 5 more mins..."  
  
};  
  
// IR expressions: 0=Surprised, 1=Unimpressed, 2=Cool  
  
const char* irMessages[3] = {  
  
    "Oh! A visitor!",  
  
    "Oh. It's you again.",  
  
    "Hey, I'm watching you"  
  
};  
  
// -----  
  
//  SENSOR READS (With filtering)  
  
// -----  
  
void readSoilMoisture() {  
  
    soilValue = analogRead(SOIL_PIN);  
  
}  
  
void readDHTSensor() {  
  
    float t = dht.readTemperature();
```

```

float h = dht.readHumidity();

if (!isnan(t)) temperature = t;

if (!isnan(h)) humidity    = h;
}

void readWaterLevel() {

    float sum = 0;

    int validSamples = 0;

    for (int i = 0; i < ULTRASONIC_READ_SAMPLES; i++) {

        digitalWrite(TRIG_PIN, LOW);

        delayMicroseconds(2);

        digitalWrite(TRIG_PIN, HIGH);

        delayMicroseconds(10);

        digitalWrite(TRIG_PIN, LOW);

        // 25ms timeout corresponds to approx 4.25 meters max distance

        long duration = pulseIn(ECHO_PIN, HIGH, 25000);

        if (duration > 0) {

            float dist = (duration * 0.0343) / 2.0;

            // Filter out physical outliers

            if (dist > 1.5 && dist < 250.0) {

                sum += dist;

```

```

        validSamples++;

    }

}

    delay(15); // Short gap between pulses to prevent echo overlap
interference

}

if (validSamples > 0) {

    waterDistance = sum / validSamples;

    // Water level is healthy if distance to water is less than
empty threshold limit

    if (waterDistance < waterThreshold) {

        consecutiveWaterFailures = 0; // Reset consecutive failures

    } else {

        consecutiveWaterFailures++; // Water is too low

    }

} else {

    waterDistance = -1; // Flag sensor measurement failure

    consecutiveWaterFailures++;

}

}

// _____

// PUMP & BUZZER

```

```
// _____

void startPump() {

    digitalWrite(RELAY_PIN, LOW); // Active-LOW relay closes switch
    to start pump

    pumpRunning = true;

}

void stopPump() {

    digitalWrite(RELAY_PIN, HIGH); // Active-LOW relay opens switch to
    stop pump

    pumpRunning = false;

}

void controlPump() {

    // If water level failure limit has been reached, protect pump by
    forcing it OFF

    bool waterEmpty = (consecutiveWaterFailures >=
    WATER_FAILURE_LIMIT);

    if (waterEmpty) {

        stopPump();

        return;

    }

    // Normal soil moisture hysteresis control

    if      (soilValue > dryThreshold    && !pumpRunning) startPump();

    else if (soilValue < moistThreshold &&  pumpRunning) stopPump();

}
```

```

void alertBuzzer() {

    bool waterEmpty = (consecutiveWaterFailures >=
WATER_FAILURE_LIMIT);

    if (waterEmpty) {

        digitalWrite(BUZZER_PIN, HIGH);

    } else {

        digitalWrite(BUZZER_PIN, LOW);

    }

}

// -----

//  DASHBOARD DISPLAY

// -----

void displayDashboard() {

    display.clearDisplay();

    display.setTextSize(1);

    display.setTextColor(WHITE);

    // Soil

    display.setCursor(0, 0);

    display.print("Soil: ");

    display.print(soilValue);

    if      (soilValue > dryThreshold)    display.print(" DRY");

    else if (soilValue < moistThreshold) display.print(" GOOD");

    else                                     display.print(" MED");

    // Temperature

```

```
display.setCursor(0, 16);

display.print("Temp: ");

display.print(temperature, 1);

display.print(" C");

// Humidity

display.setCursor(0, 32);

display.print("Hum:  ");

display.print(humidity, 1);

display.print("%");

// Water level

display.setCursor(0, 48);

display.print("Water: ");

if (consecutiveWaterFailures >= WATER_FAILURE_LIMIT) {

    if (waterDistance == -1) {

        display.print("No Echo!");

    } else {

        display.print("EMPTY!");

    }

} else {

    if (waterDistance == -1) {

        display.print("Calc...");

    } else {

        display.print(waterDistance, 1);

    }

}
```

```

        display.print("cm");
    }

}

// Pump status

display.setCursor(90, 48);

display.print(pumpRunning ? "PMP:ON" : "PMP:OF");

display.display();

}

// -----

//  EYE DRAWING

// -----

#define EL_X 38

#define ER_X 90

#define E_Y 28

#define E_R 14

void drawGrumpy() {

    display.clearDisplay();

    display.fillRoundRect(EL_X - E_R, E_Y - 4, E_R * 2, 8, 3, WHITE);

    display.fillRoundRect(ER_X - E_R, E_Y - 4, E_R * 2, 8, 3, WHITE);

    display.drawLine(EL_X - E_R, E_Y - 12, EL_X + E_R, E_Y - 8,
WHITE);

    display.drawLine(ER_X - E_R, E_Y - 8, ER_X + E_R, E_Y - 12,
WHITE);

    display.display();

```



```

}

void drawDizzy() {

    display.clearDisplay();

    display.drawLine(EL_X - 8, E_Y - 8, EL_X + 8, E_Y + 8, WHITE);

    display.drawLine(EL_X + 8, E_Y - 8, EL_X - 8, E_Y + 8, WHITE);

    display.drawCircle(EL_X, E_Y, E_R, WHITE);

    display.drawLine(ER_X - 8, E_Y - 8, ER_X + 8, E_Y + 8, WHITE);

    display.drawLine(ER_X + 8, E_Y - 8, ER_X - 8, E_Y + 8, WHITE);

    display.drawCircle(ER_X, E_Y, E_R, WHITE);

    display.display();

}

void drawSleepy() {

    display.clearDisplay();

    display.drawCircle(EL_X, E_Y, E_R, WHITE);

    display.fillRect(EL_X - E_R - 1, E_Y - E_R - 1, E_R * 2 + 2, E_R +
2, BLACK);

    display.drawFastHLine(EL_X - E_R, E_Y, E_R * 2, WHITE);

    display.drawCircle(ER_X, E_Y, E_R, WHITE);

    display.fillRect(ER_X - E_R - 1, E_Y - E_R - 1, E_R * 2 + 2, E_R +
2, BLACK);

    display.drawFastHLine(ER_X - E_R, E_Y, E_R * 2, WHITE);

    display.display();

}

void drawSurprised() {

```

```

display.clearDisplay();

display.drawCircle(EL_X, E_Y, E_R + 2, WHITE);

display.fillCircle(EL_X, E_Y, 5, WHITE);

display.drawCircle(ER_X, E_Y, E_R + 2, WHITE);

display.fillCircle(ER_X, E_Y, 5, WHITE);

display.drawFastHLine(EL_X - E_R, E_Y - E_R - 5, E_R * 2, WHITE);

display.drawFastHLine(ER_X - E_R, E_Y - E_R - 5, E_R * 2, WHITE);

display.display();
}

void drawUnimpressed() {

display.clearDisplay();

display.drawCircle(EL_X, E_Y, E_R, WHITE);

display.fillRect(EL_X - E_R - 1, E_Y - E_R - 1, E_R * 2 + 2,
(int)(E_R * 1.2), BLACK);

display.drawFastHLine(EL_X - E_R, E_Y - 4, E_R * 2, WHITE);

display.drawCircle(ER_X, E_Y, E_R, WHITE);

display.fillRect(ER_X - E_R - 1, E_Y - E_R - 1, E_R * 2 + 2,
(int)(E_R * 1.2), BLACK);

display.drawFastHLine(ER_X - E_R, E_Y - 4, E_R * 2, WHITE);

display.fillCircle(EL_X, E_Y + 4, 3, WHITE);

display.fillCircle(ER_X, E_Y + 4, 3, WHITE);

display.display();
}

void drawCool() {

```

```

display.clearDisplay();

display.drawCircle(EL_X, E_Y, E_R, WHITE);

display.fillRect(EL_X - E_R - 1, E_Y - E_R - 1, E_R * 2 + 2, E_R /
2, BLACK);

display.drawFastHLine(EL_X - E_R, E_Y - E_R / 2, E_R * 2, WHITE);

display.fillCircle(EL_X, E_Y + 2, 4, WHITE);

display.drawCircle(ER_X, E_Y, E_R, WHITE);

display.fillRect(ER_X - E_R - 1, E_Y - E_R - 1, E_R * 2 + 2, E_R /
2, BLACK);

display.drawFastHLine(ER_X - E_R, E_Y - E_R / 2, E_R * 2, WHITE);

display.fillCircle(ER_X, E_Y + 2, 4, WHITE);

display.display();
}

void drawEyes(String trigger, int exprIdx) {

    if (trigger == "touch") {

        switch (exprIdx) {

            case 0: drawGrumpy();      break;

            case 1: drawDizzy();       break;

            case 2: drawSleepy();      break;

        }

    } else {

        switch (exprIdx) {

            case 0: drawSurprised();   break;

            case 1: drawUnimpressed(); break;

        }

    }

}

```

```

        case 2: drawCool();      break;

    }

}

}

// -----

//  TEXT DISPLAY

// -----

void displayMessage(String msg) {

    display.clearDisplay();

    display.setTextSize(1);

    display.setTextColor(WHITE);

    int16_t x1, y1;

    uint16_t w, h;

    display.getTextBounds(msg, 0, 0, &x1, &y1, &w, &h);

    int cx = (SCREEN_WIDTH - w) / 2;

    int cy = (SCREEN_HEIGHT - h) / 2;

    display.setCursor(cx, cy);

    display.print(msg);

    display.display();

}

// -----

//  TRIGGER EYE SEQUENCE

// -----

```

```

void triggerSequence(String trigger) {

    if (eyeActive) return;

    eyeActive      = true;

    showingEyes    = true;

    showingText    = false;

    currentTrigger = trigger;

    eyePhaseStart = millis();

    if (trigger == "touch") {

        currentExprIdx = nextCard(touchDeck);

        currentMessage = String(touchMessages[currentExprIdx]);

    } else {

        currentExprIdx = nextCard(irDeck);

        currentMessage = String(irMessages[currentExprIdx]);

    }

    drawEyes(currentTrigger, currentExprIdx);

}

// -----

//  UPDATE EYE SEQUENCE (called every loop)

// -----

void updateEyeSequence() {

    if (!eyeActive) return;

    unsigned long elapsed = millis() - eyePhaseStart;

    if (showingEyes && elapsed >= EYE_DURATION) {

```

```

    showingEyes = false;

    showingText = true;

    eyePhaseStart = millis();

    displayMessage(currentMessage);

    return;
}

if (showingText && elapsed >= TEXT_DURATION) {

    showingEyes = false;

    showingText = false;

    eyeActive    = false;


    // Activate cooldown lockouts

    lastAnimationEndTime = millis();

    cooldownActive = true;


    // Instantly restore dashboard display when animation is
complete

    displayDashboard();

}

}

// -----

// TOUCH & IR POLLING (With active levels & debounce)

// -----

bool lastTouchState = !TOUCH_ACTIVE_STATE; // Initialize as inactive

```

```

bool lastIRState    = !IR_ACTIVE_STATE;    // Initialize as inactive

void handleTouch() {

    bool touchState = digitalRead(TOUCH_PIN);

    unsigned long now = millis();

    // Edge detection: trigger on active state transition

    if (touchState == TOUCH_ACTIVE_STATE && lastTouchState !=
TOUCH_ACTIVE_STATE) {

        // Only trigger if idle, not in cooldown lockout, and debounce
elapsed

        if (!eyeActive && !cooldownActive && (now - lastTouchTime >=
TOUCH_DEBOUNCE_MS)) {

            lastTouchTime = now;

            triggerSequence("touch");

        }

    }

    lastTouchState = touchState;
}

void handleIR() {

    bool irState = digitalRead(IR_PIN);

    unsigned long now = millis();

    // Edge detection: trigger on active state transition

    if (irState == IR_ACTIVE_STATE && lastIRState != IR_ACTIVE_STATE)
{

```

```

        // Only trigger if idle, not in cooldown lockout, and debounce
        elapsed

        if (!eyeActive && !cooldownActive && (now - lastIRTime >=
IR_DEBOUNCE_MS)) {

            lastIRTime = now;

            triggerSequence("ir");

        }

    }

    lastIRState = irState;

}

// -----

//  DEBUG SERIAL LOGGER

// -----

void logStatus() {

    Serial.print("--- SYSTEM STATUS --- [");

    Serial.print(millis() / 1000);

    Serial.println("s]");

    Serial.print("Soil Moisture Value : ");

    Serial.print(soilValue);

    Serial.print(" (Dry Threshold: ");

    Serial.print(dryThreshold);

    Serial.print(", Moist Threshold: ");

    Serial.print(moistThreshold);

```



```
Serial.println("");

Serial.print("Water Tank Distance : ");

if (waterDistance == -1) {

    Serial.print("No Echo! / Failure (Count: ");

    Serial.print(consecutiveWaterFailures);

    Serial.println("");

} else {

    Serial.print(waterDistance, 1);

    Serial.print(" cm (Threshold: ");

    Serial.print(waterThreshold);

    Serial.print(" cm, Empty Failures: ");

    Serial.print(consecutiveWaterFailures);

    Serial.println("");

}

Serial.print("Air Temperature      : ");

Serial.print(temperature, 1);

Serial.println(" C");

Serial.print("Air Humidity          : ");

Serial.print(humidity, 1);

Serial.println(" %");

Serial.print("Water Pump State      : ");

Serial.println(pumpRunning ? "RUNNING (ON)" : "STOPPED (OFF)");

Serial.print("Animation State        : ");
```

```

    if (eyeActive) {

        Serial.print("ACTIVE (Trigger: ");

        Serial.print(currentTrigger);

        Serial.println(")");

    } else if (cooldownActive) {

        Serial.print("COOLDOWN LOCKOUT (Remaining: ");

        Serial.print((COOLDOWN_DURATION - (now_diff())) / 1000.0, 1);

        Serial.println("s");

    } else {

        Serial.println("READY / IDLE (Showing Dashboard)");

    }

    Serial.println("_____");

}

// Helper to calculate cooldown remaining time correctly

unsigned long now_diff() {

    return millis() - lastAnimationEndTime;

}

// _____

//  SETUP

// _____

void setup() {

    Serial.begin(115200);

    Serial.println("Booting Smart Plant System...");

```

```

    pinMode(RELAY_PIN, OUTPUT); digitalWrite(RELAY_PIN, HIGH); //
Active-LOW relay starts OFF (HIGH)

    pinMode(TRIG_PIN, OUTPUT);

    pinMode(ECHO_PIN, INPUT);

    pinMode(BUZZER_PIN, OUTPUT); digitalWrite(BUZZER_PIN, LOW);


    // Set up inputs. Enabling internal pullup for IR to prevent
floating if open-drain/disconnected.

    pinMode(IR_PIN, INPUT_PULLUP);

    pinMode(TOUCH_PIN, INPUT);

    dht.begin();

    if (!display.begin(SSD1306_SWITCHCAPVCC, 0x3C)) {

        Serial.println("OLED not found - check wiring!");

        while (true);

    }

    display.clearDisplay();

    display.display();

    // Wait for sensor voltage rails to stabilize after boot to
prevent transient false reads

    Serial.println("Waiting 1.5s for sensor voltages to
stabilize...");

    delay(1500);

    // Seed RNG using floating ADC input

    randomSeed(analogRead(35));

    // Initialize decks

```

```

    touchDeck.lastPlayed = -1; shuffleDeck(touchDeck);

    irDeck.lastPlayed    = -1; shuffleDeck(irDeck);

    // Initialize input states after stabilization to ensure edge
    trigger accuracy

    lastIRState = digitalRead(IR_PIN);

    lastTouchState = digitalRead(TOUCH_PIN);

    // Perform first sensor reads

    readDHTSensor();

    readSoilMoisture();

    readWaterLevel();

    controlPump();

    alertBuzzer();

    displayDashboard();

    Serial.println("Initialization complete! Running loop...");

    logStatus();

}

// _____

//  LOOP

// _____

void loop() {

    unsigned long now = millis();

    // Manage animation cooldown lockout duration

    if (cooldownActive) {

```

```

    if (now - lastAnimationEndTime >= COOLDOWN_DURATION) {

        cooldownActive = false;

        Serial.println("Dashboard cooldown lockout cleared. Sensor
triggers re-enabled.");

    }

}

// Sensor reads & dashboard display update (Runs every 2 seconds)

if (now - lastSensorRead >= SENSOR_INTERVAL) {

    lastSensorRead = now;

    readSoilMoisture();

    readWaterLevel();

    controlPump();

    alertBuzzer();

    if (!eyeActive) {

        displayDashboard();

    }

    // Output full diagnostic status to Serial Monitor

    logStatus();

}

// DHT sensor reading on its own 4s timer

if (now - lastDHTRead >= DHT_INTERVAL) {

    lastDHTRead = now;

    readDHTSensor();

    if (!eyeActive) {

```

```

        displayDashboard();

    }

}

// Poll Touch and IR sensors with debounced edge detection

handleTouch();

handleIR();

// Eye sequence animation state machine

updateEyeSequence();

}

```

Blynk IoT Implementation:

```

// — Blynk config — fill in your credentials —

#define BLYNK_TEMPLATE_ID   "XXXXXXX"

#define BLYNK_TEMPLATE_NAME "smart plant system"

#define BLYNK_AUTH_TOKEN    "XXXXXXXXX"


#include <WiFi.h>

#include <BlynkSimpleEsp32.h>

#include <DHT.h>


// _____

//  Wi-Fi Credentials

// _____

```

```

char ssid[]      = "SSID_name";

char password[] = "Password";


// -----

//  PIN DEFINITIONS

// -----

#define SOIL_PIN      34

#define RELAY_PIN     19

#define TRIG_PIN      13

#define ECHO_PIN      12

#define BUZZER_PIN    27

#define DHTPIN        18

#define DHTTYPE       DHT11

#define IR_PIN        23

#define TOUCH_PIN     32    // GPIO4


// -----

//  THRESHOLDS

// -----

#define DRY_THRESHOLD    3000

#define MOIST_THRESHOLD  1200

#define WATER_THRESHOLD  8    // cm - above this = LOW

#define TANK_MED_CM      5    // cm - between 5-8 = MEDIUM, below 5 =
HIGH

```

```
#define TOUCH_THRESHOLD 40
```

```
// _____
```

```
// DHT
```

```
// _____
```

```
DHT dht(DHTPIN, DHTTYPE);
```

```
// _____
```

```
// SENSOR VALUES
```

```
// _____
```

```
int    soilValue      = 0;
```

```
float  temperature     = 0.0;
```

```
float  humidity        = 0.0;
```

```
float  waterDistance   = 0.0;
```

```
bool   pumpRunning     = false;
```

```
// _____
```

```
// BLYNK TIMER
```

```
// _____
```

```
BlynkTimer timer;
```

```
// _____
```

```
// SENSOR READS
```



```
// _____  
  
void readSoilMoisture() {  
  
    soilValue = analogRead(SOIL_PIN);  
  
}  
  
void readDHTSensor() {  
  
    float t = dht.readTemperature();  
  
    float h = dht.readHumidity();  
  
    if (!isnan(t)) temperature = t;  
  
    if (!isnan(h)) humidity    = h;  
  
}  
  
void readWaterLevel() {  
  
    digitalWrite(TRIG_PIN, LOW);  
  
    delayMicroseconds(2);  
  
    digitalWrite(TRIG_PIN, HIGH);  
  
    delayMicroseconds(10);  
  
    digitalWrite(TRIG_PIN, LOW);  
  
    long duration = pulseIn(ECHO_PIN, HIGH, 30000);  
  
    waterDistance = (duration == 0) ? -1 : (duration * 0.034) / 2.0;  
  
}  
  
// _____
```

```

// PUMP CONTROL

// -----

void startPump() { digitalWrite(RELAY_PIN, LOW); pumpRunning = true; }

void stopPump() { digitalWrite(RELAY_PIN, HIGH); pumpRunning = false; }


void controlPump() {

    bool waterOk = (waterDistance != -1 && waterDistance < WATER_THRESHOLD);

    if (!waterOk) { stopPump(); return; }

    if (soilValue > DRY_THRESHOLD && !pumpRunning) startPump();

    else if (soilValue < MOIST_THRESHOLD && pumpRunning) stopPump();

}


// -----

// BUZZER

// -----

void alertBuzzer() {

    if (waterDistance == -1 || waterDistance >= WATER_THRESHOLD)

        digitalWrite(BUZZER_PIN, HIGH);

    else

        digitalWrite(BUZZER_PIN, LOW);

}


// -----

```

```
// SOIL STATUS (0=Good, 1=Medium, 2=Dry)

// -----

int getSoilStatus() {

    if      (soilValue > DRY_THRESHOLD)    return 2; // Dry

    else if (soilValue < MOIST_THRESHOLD) return 0; // Good

    else                                     return 1; // Medium

}

// -----

// TANK STATUS (0=OK, 1=LOW)

// -----

int getTankStatus() {

    if (waterDistance == -1 || waterDistance >= WATER_THRESHOLD) return 1;
// LOW

    return 0; // OK

}

// -----

// SEND TO BLYNK - every 2s

// -----

void sendSensorData() {

    readSoilMoisture();

    readWaterLevel();

    controlPump();

}
```

```
    alertBuzzer();

    // Send to Blynk virtual pins

    Blynk.virtualWrite(V0, soilValue);

    Blynk.virtualWrite(V1, waterDistance == -1 ? 0 : waterDistance);

    Blynk.virtualWrite(V4, pumpRunning ? 1 : 0);

    Blynk.virtualWrite(V5, getSoilStatus());

    Blynk.virtualWrite(V6, getTankStatus());


    // Debug

    Serial.print("Soil: "); Serial.print(soilValue);

    Serial.print(" | Water: "); Serial.print(waterDistance);

    Serial.print("cm | Pump: "); Serial.print(pumpRunning ? "ON" : "OFF");

    Serial.print(" | Soil Status: "); Serial.print(getSoilStatus());

    Serial.print(" | Tank Status: "); Serial.println(getTankStatus());
}


// _____

//  SEND DHT TO BLYNK — every 4s

//  _____

void sendDHTData() {

    readDHTSensor();

    Blynk.virtualWrite(V2, temperature);
```

```
Blynk.virtualWrite(V3, humidity);

Serial.print("Temp: "); Serial.print(temperature);

Serial.print("C | Humidity: "); Serial.println(humidity);
}

// _____

//  SETUP

// _____

void setup() {

  Serial.begin(115200);

  pinMode(RELAY_PIN,  OUTPUT); digitalWrite(RELAY_PIN,  HIGH);

  pinMode(TRIG_PIN,  OUTPUT);

  pinMode(ECHO_PIN,  INPUT);

  pinMode(BUZZER_PIN, OUTPUT); digitalWrite(BUZZER_PIN, LOW);

  pinMode(IR_PIN,    INPUT);

  dht.begin();

  // Connect to Blynk (handles Wi-Fi connection internally)

  Blynk.begin(BLYNK_AUTH_TOKEN, ssid, password);
```

```
// Timer setup

timer.setInterval(2000L, sendSensorData);

timer.setInterval(4000L, sendDHTData);


Serial.println("=== Smart Plant Blynk Ready ===");

}


// -----
//  LOOP
// -----

void loop() {

  Blynk.run();

  timer.run();

}
```

WORKING

ESP32 Implementation

The ESP32 serves as the central controller of the Smart Plant Monitoring and Watering System. It interfaces with the soil moisture sensor, ultrasonic water level sensor, DHT11 temperature and humidity sensor, relay-controlled water pump, buzzer, OLED display, IR sensor, and touch sensor.

The soil moisture sensor continuously measures the moisture content of the soil. The measured value is compared against two predefined thresholds:

- Moist Threshold = 1800
- Dry Threshold = 3000

When the soil moisture reading exceeds 3000, the soil is classified as dry and the ESP32 activates the relay, turning the water pump ON. Irrigation continues until the moisture reading falls below 1800, indicating that sufficient water has been supplied to the plant. The use of separate ON and OFF thresholds prevents rapid switching of the pump due to minor fluctuations in sensor readings.

The ultrasonic sensor monitors the water level inside the storage tank. The distance between the sensor and the water surface is measured periodically. A distance below 8 cm indicates that sufficient water is available, whereas a distance above 8 cm indicates a low-water condition. To improve reliability, the system averages three ultrasonic measurements and employs a failure-count mechanism. The pump is stopped and the buzzer is activated only after three consecutive low-water or failed readings, preventing false alarms caused by temporary sensor disturbances.

The DHT11 sensor periodically measures ambient temperature and humidity, allowing environmental conditions around the plant to be monitored in real time.

An OLED display is used as the primary local user interface. Under normal operation, the OLED continuously displays a dashboard containing:

- Soil moisture value and status (GOOD, MED, or DRY)
- Temperature
- Humidity

- Water tank level
- Pump status (ON/OFF)

In addition to displaying system parameters, the OLED incorporates an interactive expression system controlled by the touch sensor and IR sensor.

When the touch sensor is activated, the OLED displays one of three animated eye expressions:

- Grumpy
- Dizzy
- Sleepy

Each expression is accompanied by a corresponding message:

- "Hey! I was asleep!"
- "Whoa... dizzy!"
- "Zzz... 5 more mins..."

Similarly, when the IR sensor detects the presence of a nearby person, the OLED displays one of three visitor-based expressions:

- Surprised
- Unimpressed
- Cool

These expressions are paired with the messages:

- "Oh! A visitor!"
- "Oh. It's you again."
- "Hey, I'm watching you"

To improve user interaction, the system employs a shuffled-deck selection algorithm. This ensures that all three expressions in a category are displayed once before any expression is repeated, creating a more natural and varied user experience.

When triggered, the selected eye expression is displayed on the OLED for 3 seconds. Afterward, the corresponding text message is displayed for 2 seconds. Once the sequence is completed, the OLED automatically returns to the sensor dashboard.

To ensure that sensor information remains visible, a 5-second cooldown period is enforced after each animation sequence. During this interval, additional touch or IR triggers are ignored and the dashboard remains continuously displayed.

Through the integration of automatic irrigation, environmental monitoring, water-level protection, and interactive OLED feedback, the ESP32 provides both intelligent plant care and an engaging user experience.

Blynk IoT Implementation

The Blynk IoT platform provides real-time remote monitoring of the system. During startup, the ESP32 connects to the configured Wi-Fi network and authenticates itself with the Blynk cloud server using the assigned authentication token.

The ESP32 updates the dashboard at regular intervals. Every 2 seconds, the system transmits:

- Soil moisture value
- Water tank level (distance in cm)
- Pump status
- Soil condition status
- Tank condition status

Every 4 seconds, the system transmits:

- Temperature
- Humidity

To make the dashboard easier to interpret, the ESP32 converts raw sensor readings into status codes before sending them to Blynk.

Soil Status (Virtual Pin V5)

The soil condition is represented as:

- 0 → Good (Soil moisture value below 1200)
- 1 → Medium (Soil moisture value between 1200 and 3000)
- 2 → Dry (Soil moisture value above 3000)

Tank Status (Virtual Pin V6)

The water tank condition is represented as:

- 0 → OK (Water distance less than 8 cm)
- 1 → LOW (Water distance greater than or equal to 8 cm or invalid sensor reading)

The dashboard also displays the real-time pump status, where LED glowing indicates pump is ON, and otherwise the pump is OFF.

By combining automatic irrigation control with cloud-based monitoring, the Blynk platform enables users to observe plant conditions, environmental parameters, pump activity, and tank status remotely through a smartphone, ensuring effective plant care with minimal manual intervention.

Demo Video Links

The demonstration video showcasing the working of the Smart Plant Monitoring & Watering System can be accessed using the link below.

GITHUB Repository Link (Images and videos):

<https://github.com/skafreen2225-ctrl/Smart-Plant-Monitoring-and-Watering-System/tree/main/videos>

The demonstration includes:

- Soil moisture monitoring
- Automatic irrigation
- Water tank level monitoring and Buzzer alerts.
- OLED dashboard operation
- Touch sensor interaction
- IR-based proximity detection
- Blynk IoT monitoring

TESTING RESULTS

The Smart Plant Monitoring & Watering System was tested under various operating conditions to verify the functionality of each module. The results confirmed that the system performed as intended and successfully integrated automation, monitoring, and user interaction features.

Test Case	Expected Result	Observed Result	Status
Soil moisture below threshold	Water pump should turn ON automatically	Pump activated and watered the plant	Pass
Soil moisture above threshold	Water pump should turn OFF	Pump stopped automatically	Pass
Water tank level sufficient	System should operate normally	Normal operation observed	Pass
Water tank level low	Buzzer should alert the user	Buzzer activated successfully	Pass
Water tank empty	Pump should not operate	Pump operation prevented	Pass
Temperature monitoring	DHT11 should display temperature values	Temperature displayed correctly on OLED and Blynk	Pass
Humidity monitoring	DHT11 should display humidity values	Humidity displayed correctly on OLED and Blynk	Pass
OLED dashboard display	Sensor values should be displayed	Real-time values displayed successfully	Pass
Touch sensor interaction	OLED expression should change	Expression changed successfully	Pass
IR proximity	OLED should	Interactive	Pass

detection	react to nearby user	response triggered successfully	
Expression randomization	Expressions should vary between interactions	Multiple expressions displayed correctly	Pass
Blynk dashboard update	Sensor values should update remotely	Values updated successfully on dashboard	Pass
PCB connectivity verification	All modules should communicate correctly	No connectivity issues observed	Pass
CAD enclosure fitting	Components should fit within enclosure	Components accommodated successfully	Pass

The testing process demonstrated reliable operation of the automatic irrigation system, sensor monitoring functions, OLED interaction features, IoT connectivity, PCB design, and enclosure layout.

CHALLENGES FACED

During the development of the Smart Plant Monitoring & Watering System, several technical and design challenges were encountered:

1. Integration of Multiple Sensors

The project required interfacing multiple sensors, including the soil moisture sensor, DHT11, HC-SR04 ultrasonic sensor, IR sensor, and touch sensor with a single ESP32. Managing GPIO assignments and ensuring proper communication between all modules required careful planning.

2. Ultrasonic Sensor Measurement Stability

The ultrasonic sensor occasionally produced inconsistent readings due to reflections and environmental factors. To improve measurement reliability, multiple readings were averaged and invalid values were filtered in software.

3. Balancing Multiple Functionalities

The project combines automation, environmental monitoring, OLED animations, user interaction, IoT connectivity, PCB design, and CAD design within a single system. Ensuring that all features operated simultaneously without interfering with one another required careful software development, testing, and debugging.

4. OLED Expression Implementation

Designing interactive OLED expressions while simultaneously displaying sensor data required careful software organization. The display had to switch smoothly between dashboard mode and expression mode without affecting system operation.

5. PCB Routing and Component Placement

While designing the PCB in EasyEDA, proper component placement and routing were necessary to avoid overlapping tracks and maintain a compact board layout. Design Rule Checks (DRC) were performed to identify and resolve routing issues.

6. Enclosure Design Constraints

The enclosure had to accommodate the plant holder, water reservoir, electronics compartment, sensors, drainage system, and user interaction components within a limited space. Achieving a functional and accessible layout required multiple design iterations in Onshape.

7. Water Management Design

Preventing water accumulation inside the plant compartment while ensuring effective irrigation was a key challenge. This was addressed through the incorporation of a sloped plant base, drainage holes, and a dedicated drainage mechanism.

8. Integration of Hardware and IoT Features

Combining local automation with remote monitoring through the Blynk platform required synchronization between sensor acquisition, pump control, OLED updates, and cloud communication. Ensuring reliable operation of all subsystems simultaneously required extensive testing and debugging.

FUTURE IMPROVEMENTS

Although the Smart Plant Monitoring & Watering System successfully achieves its intended objectives, several enhancements can be implemented in future versions:

1. Mobile Notifications

Push notifications can be integrated to alert users when the water tank is low, soil moisture reaches critical levels, or environmental conditions become unsuitable for plant growth.

2. Plant Growth Monitoring

Future versions can incorporate image-based plant growth monitoring using a camera module. Periodic images can be captured and analyzed to track plant growth, detect abnormalities, and provide additional insights into plant health.

3. Multiple Plant Support

The system can be expanded to monitor and irrigate multiple plants simultaneously using additional moisture sensors and watering channels.

4. Advanced Plant Health Monitoring

Additional sensors, such as light intensity and soil nutrient sensors, can be integrated to provide a more comprehensive assessment of plant health.

5. Improved User Interface

A more advanced web or mobile dashboard can be developed to provide historical data visualization, analytics, and remote configuration options.

6. AI-Based Plant Care Recommendations

Machine learning techniques can be used to analyze environmental data and provide personalized watering schedules and plant care recommendations.

7. Compact Product Development

Future versions can incorporate custom-designed PCBs and optimized enclosure designs to reduce overall size and improve the product's appearance and manufacturability.

8. Automatic Water Refilling System

An automatic water refilling mechanism can be integrated to maintain the reservoir water level without manual intervention. This would further reduce maintenance requirements and improve system autonomy.